Formal Verification of Information Flow and Side-Channel Leaks via Leaky Assertions in TLA+/TLC

Experimental Evaluation Report and Model Analysis

Formal Methods and Security Research Group

October 14, 2026

1 Experimental Evaluation Summary

Table 1 summarizes the model checking statistics obtained from TLC runs across all six benchmark suites. The state-space exploration metrics reflect breadth-first model checking on each specification.

Benchmark Program	No. of Threads (n)	Total States	Depth	TLC Time (ms)	Leak Found? (Yes/No)
`ConcurrentLeak`	2	44	5	571	**Yes**
`InternalTiming`	2	20	10	706	**Yes**
`Transformed`	2	22	10	586	**No**
`ModSqLeaks`	1	322	21	466	**Yes**
`WebServiceLeak`	1	17	5	485	**Yes**
`SeqLock`	3	3	2	729	**Yes**

Table 1: Model checking results across all benchmark programs evaluated in TLC.

Note on Transformed and Mitigated Programs:

- In `InternalTimingTransformed`, the branch-padding mitigation successfully eliminates execution-time variance ($\Delta = 9$ across all inputs), resulting in **No** leak found (all non-interference invariants pass).
- For `WebServiceLeak` and `SeqLock`, the table presents the vulnerable baseline configuration (`Fixed = FALSE`). When the mitigation toggle is engaged (`Fixed = TRUE`), the leak is completely eliminated, and TLC reports 0 violations across all reachable states.

2 Reports on the Programs

2.1 ConcurrentLeak (`ConcurrentLeaks/ConcurrentLeak.tla`)

2.1.1 1. How the Program is Leaking Information

The program models two concurrent processes, T_1 and T_2, synchronizing via a shared binary semaphore (`sem`) and sharing variable v. Process T_1 unconditionally enters a critical section, emits public output event `"a"`, increments v, emits public event `"b"`, and releases the semaphore. Process T_2 emits public entry event `"c"`, records auxiliary entry timestamp $T_{\text{start}} := t$, and then conditionally executes an internal critical section based on the secret:

- When $h = 1$: T_2 contends for the semaphore, waits for T_1 to finish, increments $v := v + 2$, and releases the semaphore before emitting exit event `"d"`.

- When $h = 0$: T_2 skips the critical section completely and immediately emits exit event `"d"`.

This introduces two distinct side-channel leaks:

1. **Timing Side-Channel**: When $h = 0$, T_2 exits fast ($\Delta \leq 3$). When $h = 1$, semaphore contention forces T_2 to serialize behind T_1, yielding prolonged elapsed execution time ($\Delta \geq 7$).
2. **Order Covert Channel**: Interleaving produces observable output event traces in sequence `out`. When $h = 1$, serialization forces the trace prefix `"acdb"`, whereas when $h = 0$, T_2 can finish earlier producing interleavings like `"cadb"` or `"cdab"`.

2.1.2 2. Detail About the Secret Variables

- **Variable**: $h \in \{0, 1\}$.
- **Classification**: Secret (high confidentiality).
- **Role**: Directly governs whether T_2 attempts to acquire the shared semaphore and access the shared variable v.

2.1.3 3. Leaky Assertions Used to Identify Leaks in TLC

TLC verifies the following invariants at the exit public statement l_7:

- **Soundness of Timing Leaky Assertion (Evaluates to TRUE)**:

$$\begin{aligned} \text{LA_Timing} &\equiv \text{Observed} \implies ((\Delta \geq \Delta_{\text{hi}}) \iff (h = 1)) \\ \text{LA_Time_Low} &\equiv (\text{Observed} \wedge \Delta \leq \Delta_{\text{lo}}) \implies (h = 0) \\ \text{LA_Time_High} &\equiv (\text{Observed} \wedge \Delta \geq \Delta_{\text{hi}}) \implies (h = 1) \end{aligned}$$

  where $\Delta_{\text{lo}} = 3$ and $\Delta_{\text{hi}} = 7$.

- **Soundness of Order Leaky Assertion (Evaluates to TRUE)**:

$$\text{LA_Trace1} \equiv (\text{Observed} \wedge \text{out} = \langle \text{"a"}, \text{"c"}, \text{"d"}, \text{"b"} \rangle) \implies (h = 1)$$

- **Leak Discovery Invariant (Violated in TLC)**:

$$\text{NoTimingLeak} \equiv \text{Observed} \implies (\Delta = \Delta_{\text{lo}})$$

  TLC refutes this property, outputting the counterexample trace where $h = 1$ causes $\Delta = 7$.

2.2 InternalTiming (`InternalTiming/InternalTiming.tla`)

2.2.1 1. How the Program is Leaking Information

This model captures the subtle interaction between secret-dependent branching and a deterministic round-robin scheduler. Two threads, T_1 and T_2, alternate execution turns:

- Thread 1 conditionally branches on secret h:
  - If $h = 1$: executes through labels $l_1 \to l_2 \to l_3 \to l_5$, requiring 3 scheduling turns before reaching public assignment $l := 1$ (`out := Append(out, "1")`).
  - If $h = 0$: executes through labels $l_1 \to l_4 \to l_5$, requiring only 2 scheduling turns before reaching l_5.
- Thread 2 executes sequentially through $m_1 \to m_2 \to m_3$ where it sets $l := 0$ (`out := Append(out, "0")`).

Because Thread 1's branch imbalance alters the scheduler turn allocation, the total global execution time across both threads differs: $\Delta = 8$ clock cycles when $h = 0$, and $\Delta = 9$ clock cycles when $h = 1$. An external observer measuring elapsed time between the public entry statement `«"START"»` and public termination `«"END"»` directly reconstructs the private branch choice.

2.2.2 2. Detail About the Secret Variables

- **Variables**: $h \in \{0, 1\}$ (primary branch secret), $h_2 \in \mathbb{N}$ (cryptographic accumulator updated via $h_2 := 2 \cdot h_2 + l$).
- **Classification**: Secret (high data).
- **Role**: Governs the execution path length in Thread 1, directly perturbing round-robin turn progression.

2.2.3 3. Leaky Assertions Used to Identify Leaks in TLC

Evaluated at process termination (`Observed`):

- **Attacker Deductive Leaky Assertions (PASS in TLC)**:

$$\begin{aligned} \text{Calib0} &\equiv (\text{Observed} \wedge h = 0) \implies \Delta = 8 \\ \text{Calib1} &\equiv (\text{Observed} \wedge h = 1) \implies \Delta = 9 \\ \text{LA_Timing} &\equiv \text{Observed} \implies ((\Delta \geq 9) \iff (h = 1)) \end{aligned}$$

- **Security Invariant (FAILS in TLC)**:

$$\text{NoTimingLeak} \equiv \text{Observed} \implies (\Delta = 8)$$

  TLC finds an execution path with $h = 1$ yielding $\Delta = 9$, disproving timing non-interference.

2.3 Transformed (`InternalTimingTransformed/Transformed.tla`)

2.3.1 1. How the Program is Leaking Information (Mitigation Analysis)

`Transformed` represents the mitigated countermeasure specification. To eliminate the internal timing covert channel caused by the branch imbalance in Thread 1, source-level *branch padding* is introduced.

- On the shorter path ($h = 0$), an auxiliary padding step is inserted:

```
l4_pad:  await turn = 1; skip; t := t + 1; turn := 2; goto l5;
```

- This balances the turn consumption: both branches now consume exactly 3 scheduler turns before reaching l_5.
- As a result, the total elapsed execution time becomes strictly invariant: $\Delta = 9$ across both $h = 0$ and $h = 1$. The side-channel leak is eliminated.

2.3.2 2. Detail About the Secret Variables

- **Variables**: $h \in \{0, 1\}$ and $h_2 \in \mathbb{N}$.
- **Classification**: Secret (high data).
- **Role**: Controls branching in Thread 1; however, both branches are now structurally balanced in scheduling turns and execution cycles.

2.3.3 3. Leaky Assertions Used to Identify Leaks in TLC

- **Security Invariant (PASSES in TLC)**:

$$\text{ConstantTime} \equiv \text{Observed} \implies (\Delta = 9)$$

$$\text{NoTimingLeak} \equiv \text{Observed} \implies (\Delta = 9)$$

  TLC checks all 20 reachable states and confirms that execution time is completely independent of h.

- **Attacker Leaky Assertion Defeat**: The previous attacker leaky assertion LA_Timing $\equiv$ Observed $\implies$ $(\Delta \geq 9 \iff h = 1)$ is invalidated, proving the attacker can no longer distinguish $h = 0$ from $h = 1$.

2.4 ModSqLeaks (`ModSqLeaks/ModSqLeaks.tla`)

2.4.1 1. How the Program is Leaking Information

This specification models the classic square-and-multiply algorithm used in modular exponentiation ($y^K \pmod N$), foundational to RSA and Diffie-Hellman implementations.

- For each bit i of the exponent K:
  - The algorithm unconditionally executes a modular squaring operation ($t := t + 1$).
  - If the secret bit $K[i] = 1$, it additionally executes a modular multiplication ($t := t + 1$).
  - If $K[i] = 0$, the multiplication is bypassed (when `Fixed = FALSE`).
- **The Leak**: Total execution time $\Delta = \text{Len}(K) + \text{HW}(K)$, where $\text{HW}(K)$ is the Hamming weight (number of 1-bits) of the secret exponent. An observer measuring execution time learns the exact Hamming weight or bit threshold of the private key.
- **Mitigation (`Fixed = TRUE`)**: A dummy multiplication is performed when $K[i] = 0$, guaranteeing strict constant-time execution ($\Delta = 2 \cdot \text{Len}(K)$).

2.4.2 2. Detail About the Secret Variables

- **Variable**: $K \in \text{Seq}(\{0, 1\})$, representing the binary representation of the cryptographic key.
- **Derived Secret**: $\text{HW}(K) = \sum_i K[i]$ (Hamming weight).
- **Classification**: Highly confidential cryptographic secret.

2.4.3 3. Leaky Assertions Used to Identify Leaks in TLC

- **Sound Leaky Assertion (Valid in TLC)**:

$$\text{LeakyAssertion} \equiv \text{Observed} \implies (\Delta \geq \text{Threshold} \iff \text{HW}(K) \geq \text{SecretThreshold})$$

  Formally validates that execution time directly reveals private key bit density.

- **Non-Interference Invariant (FAILS under vulnerable mode)**:

$$\text{ConstantTime} \equiv \text{Observed} \implies (\Delta = \text{ExpectedConstantTime})$$

  Under `Fixed = FALSE`, TLC violates this invariant and returns traces of varying key weights.

2.5 WebServiceLeak (`WebServiceLeak/WebServiceLeak.tla`)

2.5.1 1. How the Program is Leaking Information

This model captures a user enumeration timing attack against a web authentication endpoint:

- A client sends a login request (`l0: out := Append(out, "REQ")`).
- The server queries the database (`Lookup: t := t + LookupCost`).
- **Vulnerable Path (`Fixed = FALSE`)**:
  - If the username does not exist (`valid_user = FALSE`), the server immediately aborts and returns (`OutputError: result := "error message"`). Total duration: $\Delta = \text{LookupCost} = 1$.
  - If the username exists (`valid_user = TRUE`), but the password is wrong, the server performs an expensive cryptographic password hash (`HashPassword: t := t + HashCost`) before returning the identical error message. Total duration: $\Delta = \text{LookupCost} + \text{HashCost} = 11$.
- Both paths return the identical generic string `"error message"`, but the response latency differs by an order of magnitude, allowing an external client to deduce whether any target username exists.
- **Mitigation (`Fixed = TRUE`)**: When the username does not exist, the server executes a dummy hash computation (`DummyHash: t := t + HashCost`), equalizing both failure paths to $\Delta = 11$.

2.5.2 2. Detail About the Secret Variables

- **Variables**: `valid_user` $\in \{\text{TRUE}, \text{FALSE}\}$, h = IF `valid_user` THEN 1 ELSE 0, and `password_match` $\in \{\text{TRUE}, \text{FALSE}\}$.
- **Classification**: Secret (confidential database metadata / account existence).
- **Role**: Governs whether the cryptographic hashing routine is bypassed.

2.5.3 3. Leaky Assertions Used to Identify Leaks in TLC

Evaluated at the HTTP response exit point (`l_exit`):

- **Attacker Deductive Leaky Assertions (PASS in TLC)**:

$$\text{LA_Time_Low} \equiv (\neg\text{Fixed} \wedge \text{Observed} \wedge \text{PublicOut} = \text{"error message"} \wedge \Delta \leq \Delta_{\text{lo}}) \implies (h = 0)$$
$$\text{LA_Time_High} \equiv (\neg\text{Fixed} \wedge \text{Observed} \wedge \text{PublicOut} = \text{"error message"} \wedge \Delta \geq \Delta_{\text{hi}}) \implies (h = 1)$$
$$\text{LA_Timing} \equiv (\neg\text{Fixed} \wedge \text{Observed} \wedge \text{PublicOut} = \text{"error message"}) \implies ((\Delta \geq \text{Threshold}) \iff (h =$$

  where $\Delta_{\text{lo}} = 1$, $\Delta_{\text{hi}} = 11$, and $\text{Threshold} = 5$.

- **Security Invariant (FAILS when vulnerable)**:

$$\text{NoTimingLeak} \equiv (\text{Observed} \wedge \text{result} = \text{"error message"}) \implies (\Delta = \text{LookupCost} + \text{HashCost})$$

  TLC finds the early abort trace where non-existent users return in $\Delta = 1$ instead of 11.

2.6 SeqLock (`SeqLock/SeqLock.tla`)

2.6.1 1. How the Program is Leaking Information

Extracted from Graeme Smith's recent work on *Information Flow Security on Persistent Memory* (2026), this benchmark models the Linux reader-writer `seqlock` mechanism running on Intel-x86 persistent memory (PM) subject to power outages.

- Shared memory locations:
  - c: sequence counter (even = idle, odd = write in progress).
  - x_1: data variable containing either low data or sensitive high secret ($L(x_1) = \text{high}$).
  - x_2: classification flag ($x_2 = 1 \iff x_1$ is high; $L(x_2) = \text{low}$).
- **The Persistency Leak (`Fixed = FALSE`)**: Stores to x_1, x_2, and c reside on different cache lines. Under weak persistency models without explicit cache flushes, stores are committed to persistent memory asynchronously. If the store to x_1 and the final even store to c persist to PM, while the store to tag x_2 lags in volatile buffers when a power outage occurs, rebooting restores an inconsistent state:

$$c \text{ is even}, \quad x_2 = 0 \text{ (flag indicates low)}, \quad x_1 = 1 \text{ (holds sensitive high secret!)}$$

  A reader executing post-reboot sees c even, reads $x_2 = 0$, concludes the data is low, reads x_1, and emits the high secret over the public low channel.
- **Mitigation (`Fixed = TRUE`)**: Inserts `flush c` after the first increment of c, and `flush x1; flush x2` before the final increment of c. This ensures c is never persisted as even unless both x_1 and x_2 are safely committed to persistent memory.

2.6.2 2. Detail About the Secret Variables

- **Variables**: $h \in \{0, 1\}$, where $h = 1$ denotes high confidential data, and $h = 0$ denotes low public data.
- **Classification**: Secret (high security classification $\Gamma_{x_1} = \text{high}$).
- **Persistent Locations**: `pm_x1` (persistent data), `pm_x2` (persistent security flag), `pm_c` (persistent sequence counter).

2.6.3 3. Leaky Assertions Used to Identify Leaks in TLC

- **Sound Leaky Assertion (PASSES in TLC)**:

  $\text{LA_PersistentLeak} \equiv (\neg\text{Fixed} \wedge \text{crashed} \wedge \text{ReaderTerminated} \wedge (\exists k \in 1..\text{Len}(\text{out}) : \text{out}[k] = 1)) \implies (h = 1)$

  Proves that an observer observing output value 1 on the low reader output channel after a crash soundly infers that the high secret was $h = 1$.
- **Value-Dependent Non-Interference Property (FAILS in vulnerable mode)**:

$$\text{NoInformationLeak} \equiv \forall k \in 1..\text{Len}(\text{out}) : (\text{out}[k] = 1) \implies (h = 0)$$

  When `Fixed = FALSE`, TLC finds the crash-and-reboot trace where the high secret ($h = 1$) is leaked through the reader's low channel, producing the counterexample. When `Fixed = TRUE`, TLC proves that `NoInformationLeak` holds across all possible interleavings.